AI651 — Deep Learning for Space, Time and Graphs (Fall 2026)

Assignment 2: Report

Code repository (must be PUBLIC at submission): https://github.com/<YOUR-USERNAME>/<YOUR-REPO> (replace with the public repository URL before submitting)

Contents: Question 1 (river forecasting proposal, six required headings); Question 2 (Motion or Memory?, Tables 1–6, Figures 1–6, Q2.1–Q2.17); AI-use disclosure (last page).

Question 1 — River Forecasting Proposal

Proposal in one paragraph. Every 10 minutes the system predicts the level at S1–S4 one hour ahead with a **graph space-time network**. The river is a directed graph (Reservoir → S1 → S2 → S3 → S4, T1 → S2, an unobserved node U → S3). Each node has a 13.5-hour history. Blocks alternate a causal temporal convolution (T) with a **travel-time-aware** message-passing step (S) along the river. Announced releases are placed on the future part of the reservoir's time axis. The forecast is the last observed level plus a learned change, with 10/50/90 % quantiles. Missing data, late data and release changes are handled in the input layer, not by retraining.

1. Architecture diagram

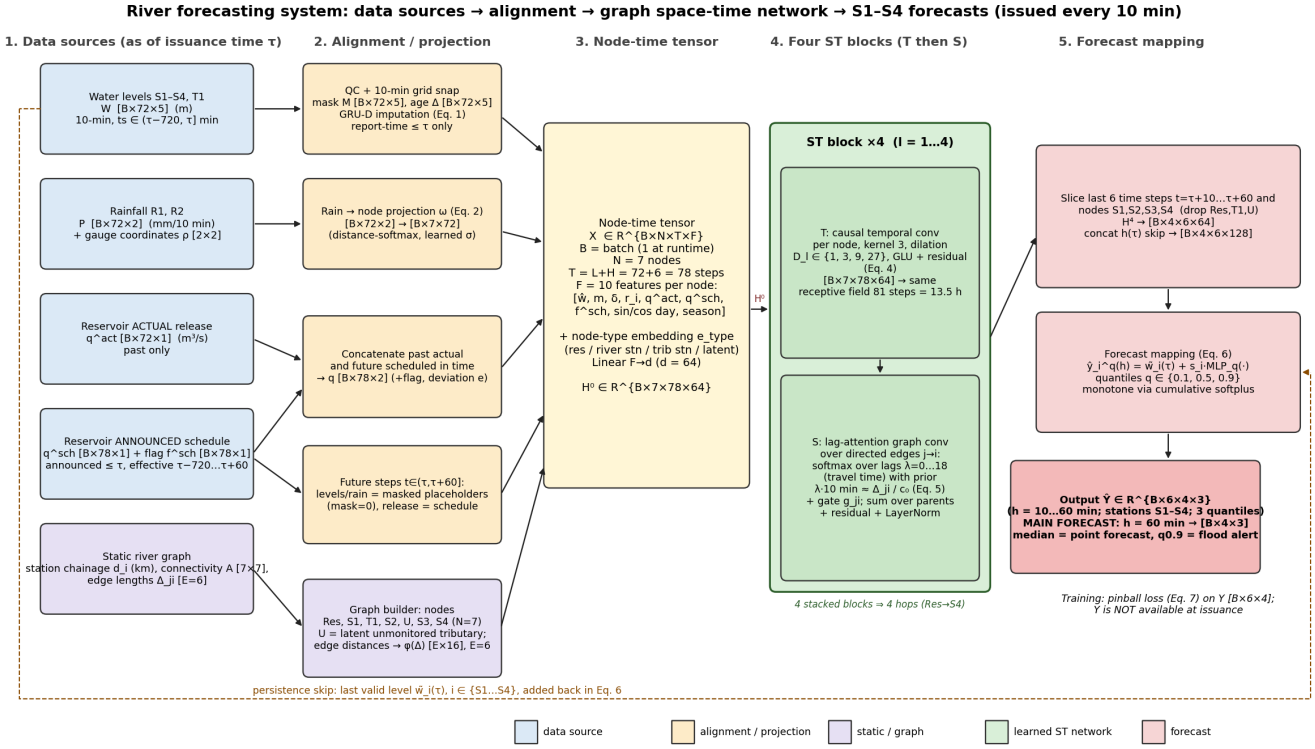


Figure Q1.1. Architecture. Sizes: B = batch (1 at run time, many in training); N = 7 graph nodes (Res, S1, T1, S2, U, S3, S4); L = 72 history steps (12 h) and H = 6 future steps (1 h), so T = 78; F = 10 input features per node; d = 64 hidden channels; E = 6 directed edges (Res→S1, S1→S2, T1→S2, S2→S3, U→S3, S3→S4); every node also keeps its own state through the $W_1 z_i$ term of Eq. 5b. **Alignment / reshaping:** (i) all series are snapped to one 10-min grid and stacked on a common time axis of length 78; (ii) the two rain gauges [B×72×2] are projected to node space [B×7×72] by the weights ω of Eq. 2, then right-padded with zeros and mask 0 to length 78; (iii) station levels [B×72×5] are scattered to the nodes (T1→T1, S1–S4→S1–S4; Res and U get zeros and their own mask); (iv) reservoir actual [B×72] and announced [B×78] releases are written on the Res node's time axis; (v) channels are concatenated to [B×7×78×10] and projected to d = 64. The ST stack keeps [B×7×78×64] throughout; the head keeps only nodes S1–S4 and the last six time positions, giving [B×4×6×64] → forecasts [B×6×4×3] (6 horizons × 4 stations × 3 quantiles).

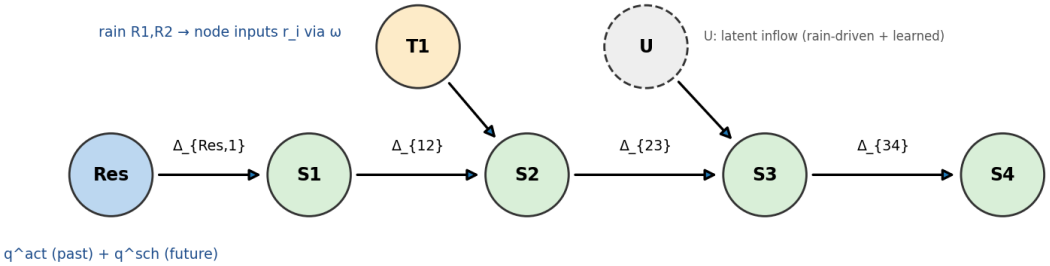


Figure Q1.2. The river as a directed graph and the number of hops from each source to S4. Because T1 joins before S2 and U before S3, four stacked ST blocks (4 hops) are enough for information from the reservoir to reach S4.

2. Assumptions

Supplied facts (not assumptions): F1 levels at S1–S4 and T1 every 10 min; F2 rainfall at R1, R2 with timestamps and gauge locations; F3 recorded reservoir releases and future release schedules announced before issuance, with announcement and effective times; F4 station locations, connectivity and distances (stations unevenly spaced); F5 a monitored tributary joins between S1 and S2, an unmonitored one between S2 and S3; F6 future rain and future levels are unavailable at issuance; F7 travel times and unmonitored inflow are not given; F8 additional information may be requested but is not guaranteed. **Everything below is an assumption.**

ID	Assumption	Why it is reasonable / how it affects the design
A1	Record length: ≥ 5 years of 10-min data (~263 000 steps) that include several flood events and wet seasons.	Typical for a national gauge archive. Allows a ~0.25 M-parameter network and a chronological train/val/test split (first 3.5 y / next 0.75 y / last 0.75 y). If the record is under 2 y, shrink d and lean on the physical priors (A6–A8).
A2	All series share one clock (UTC) with error < 1 min; a level is an instantaneous reading at its timestamp; rainfall is an accumulation over the previous 10 min.	Standard telemetry practice. Series are snapped to a common 10-min grid with ±2 min tolerance; no sub-grid interpolation model is needed.
A3	Reporting delay is usually ≤ 5 min but sometimes 30 min to several hours; each record also has an arrival (ingest) time.	Needed to know what was available at issuance. Drives the as-of snapshot, the 'age' feature δ and delay simulation in training (§5).
A4	1–5 % of readings are missing at random, outages of up to ~2 h occur, multi-day outages are rare; spikes and flat-lines are flagged by QC.	Sets the random-masking rates used in training and the choice of masks + decay imputation (Eq. 1).
A5	Level noise σ ≈ 1–2 cm, 1 cm resolution, no datum shifts within the record.	Below the forecast skill we target, so no explicit sensor-noise model; the pinball loss is robust to outliers.
A6	Travel times: reservoir → S1 under ~1 h; each later reach 20 min – 3 h; end-to-end reservoir → S4 ≲ 8 h; shorter at high flow (wave celerity rises with stage).	Not supplied (F7), so the model learns them. It fixes L = 12 h (> 1.5× the longest path), the lag window Λ = 18 steps (3 h) per edge, and a state-dependent celerity in Eq. 5. If a reach is slower than 3 h the lag window is widened.
A7	Flow is one-dimensional and downstream; backwater from downstream onto upstream stations is negligible (no tide, no downstream dam).	Justifies a *directed* graph: messages travel only downstream. If backwater existed we would add upstream edges.
A8	Mass balance at confluences: $Q_{down} \approx Q_{up} + Q_{trib} + Q_{lateral}$; at each station level is a monotone, smooth function of discharge (unknown stage–discharge curve).	Messages entering S2 and S3 are *summed* (Eq. 5) rather than averaged; non-negative gates keep contributions monotone.
A9	T1 is gauged close to its confluence (travel time < 30 min), so T1 is effectively the tributary inflow joining between S1 and S2.	Gives a short edge T1 → S2 with a small expected lag.
A10	The unmonitored tributary U has a catchment responding to rain like T1 (lag 1–6 h) plus a slowly varying base flow; it enters between S2 and S3, so it affects S3 and S4 only.	Modelled as a latent node whose state comes from rainfall and a learned base level; the only information about it is the mismatch between S2 and S3, so S3/S4 get wider intervals (§6).
A11	R1 and R2 give 10-min accumulations in mm; their coordinates are known but two gauges only roughly represent the catchment. Rain reaches the river with a lag; a small part is direct channel runoff.	Rain is a node feature from distance weights (Eq. 2); because future rain is unknown (F6), only rain that has already fallen can help the 1-hour forecast.
A12	Releases are announced typically 1–24 h ahead as planned discharge with an effective time; a newer announcement replaces an older one for the same effective time; actual release is metered every 10 min. Where nothing is announced for a future step, the current release continues.	Defines the schedule channel with an 'announced' flag f^{sch} (0 = default persistence, 1 = announced), so the model knows how much to trust it (§5).

ID	Assumption	Why it is reasonable / how it affects the design
A13	Actual release follows the schedule within ~10 %, with gate-actuation delays up to 30 min; unannounced emergency changes are rare but possible.	Past actual and future scheduled releases are separate channels and a deviation feature $e = q^{act} - q^{sch}$ is kept for the recent past (§5, iv).
A14	Channel geometry and gauge datums are stable; seasonal roughness (vegetation) changes slowly.	Season encodings capture the slow change; retrain yearly or after a major flood reshapes the channel.
A15	A forecast is issued on every 10-min tick τ after a 2-min collection cut-off; the target is the level on the grid at τ + 60 min.	Fixes the task in §3 and the as-of rule: only records that have arrived by the cut-off are used.
A16	A CPU server runs inference; training is offline and may use a GPU.	Model stays small (§5 feasibility).

3. Forecasting task and data specification

Issuance time. τ is any 10-minute grid time. The forecast issued at τ uses only records whose arrival time is ≤ τ + 2 min (A15), and announcements made ≤ τ. **History window.** L = 72 steps (12 h): grid times τ−710, …, τ min (A6: the longest path is under 8 h). **Horizon.** H = 6 steps = 60 min. The network produces h = 1, …, 6 (τ+10, …, τ+60 min) as auxiliary outputs and the **main product is h = 6**. **Outputs.** Water level in metres at S1, S2, S3, S4 at τ + 60 min as the 10 %, 50 % and 90 % quantiles; the median is the point forecast, the 90 % quantile feeds flood warnings.

Item	Symbol and shape	Axes	Timestamps / coverage (relative to τ)	At issuance?
Water levels S1–S4, T1	$W \in R^{B\times 72\times 5}$ (m)	batch; time (10-min grid); station (S1,S2,S3,S4,T1)	τ−710 … τ min	Yes, only records arrived by τ
Observation mask, age	$M \in \{0,1\}^{B\times 72\times 5}$, $\Delta \in R^{B\times 72\times 5}$ (min)	as W	τ−710 … τ	Yes (derived from arrival log)
Rainfall R1, R2	$P \in R^{B\times 72\times 2}$ (mm per 10 min); gauge coordinates $\rho \in R^{2\times 2}$	batch; time; gauge	(τ−720, τ] accumulations, stamped at interval end	Past only. Future rain: NOT available (F6)
Reservoir release, actual	$q^{act} \in R^{B\times 72}$ (m³/s)	batch; time	τ−710 … τ	Yes
Reservoir release, announced	$q^{sch} \in R^{B\times 78}$, flag $f^{sch} \in \{0,1\}^{B\times 78}$	batch; time (72 past + 6 future)	τ−710 … τ+60; effective times; announced ≤ τ	Yes (announcement time ≤ τ only)
Static graph	chainage $d \in R^{7}$ (km), adjacency $A \in \{0,1\}^{7\times 7}$, edge length $\Delta \in R^{6}$	node; edge	constant	Yes
Calendar	sin/cos of day and of year $\in R^{B\times 78\times 4}$	batch; time; 4 channels	τ−710 … τ+60	Yes (known in advance)
Node-time tensor (model input)	$X \in R^{B\times 7\times 78\times 10}$	batch; node (Res,S1,T1,S2,U,S3,S4); time; feature	τ−710 … τ+60 (future slots masked)	Yes: built from the rows above
Forecast (output)	$\hat{Y} \in R^{B\times 6\times 4\times 3}$, main: $\hat{Y}[:,5] \in R^{B\times 4\times 3}$ (m)	batch; horizon h=1…6; station S1–S4; quantile	τ+10 … τ+60 min	Produced at τ
Target (training only)	$Y \in R^{B\times 6\times 4}$ with mask	batch; horizon; station	τ+10 … τ+60 min	NOT available at τ

Alignment. (a) *Sampling times*: every series is snapped to the same 10-min grid; a level is the reading nearest to the grid time within ±2 min, rain is summed over (t−10, t], and the release schedule is a step function evaluated by its *effective* time (zero-order hold). (b) *Locations*: stations are not resampled in space. They sit on a graph with their true chainage and edge lengths, so unequal spacing is represented by the edge feature φ(Δ) and by the travel-time prior in Eq. 5. (c) *Gauges to nodes*: R1 and R2 are mapped to nodes by distance weights (Eq. 2). (d) *What is known at τ*: all past levels, rain, actual releases; the schedule for τ+10 … τ+60 if announced by τ; the graph and calendar. *Not known*: any level or rain after τ, and any later announcement. Training windows are built the same way (as-of τ), so nothing from the future leaks into the inputs. **Batch axis**: in training each sample is one issuance time τ (stride 1, shuffled); at run time B = 1.

4. Model computations and design justification

Notation. i, j index nodes; t a grid step (10 min); $w_i(t)$ a level, $m_i(t)$ its mask (1 = valid), $\delta_i(t)$ minutes since the last valid value; $P_k(t)$ rain at gauge k; $q^{act}(t)$, $q^{sch}(t)$, $f^{sch}(t)$ release channels; $h_i^l(t) \in R^d$ the hidden state after block l; Δ_{ji} the river distance of edge j→i; □(i) the parents of node i; 10λ minutes is a lag. Only the four river stations carry a level to forecast.

$$\hat{w}_i(t) = m_i(t)\, w_i(t) + (1 - m_i(t))[\gamma_i(t)\, w_i^{\text{last}}(t) + (1 - \gamma_i(t))\, \mu_i], \quad \gamma_i(t) = \exp(-\text{softplus}(a_i\, \delta_i(t) + b_i)) \qquad (1)$$

Missing levels decay from the last valid value towards the station mean μ_i (as in GRU-D); the mask and age are also inputs. *Physical assumption:* none. *Learned:* a_i, b_i (how fast a gap loses information).

$$r_i(t) = \sum_{k=1}^{2} \omega_{ik} P_k(t), \qquad \omega_{ik} = \mathrm{softmax}_k(-\|x_i - \rho_k\|/s) \tag{2}$$

Rain reaches a node with weights that decrease with the distance between the node and the gauge (initialised from coordinates, then fine-tuned; free logits for the latent node U). The node input is $x_i(t) = [\hat{w}_i, m_i, \delta_i/\Delta_0, r_i, q^{act}, q^{sch}, f^{sch}$, 4 calendar terms] (release channels are non-zero only on Res, level channels only on S1–S4 and T1). Then $h_i^0(t) = W_{in}\, x_i(t) + e_{type(i)}$, with a learned embedding for the node type (reservoir, river station, tributary station, latent).

$$z_i^l(t) = h_i^{l-1}(t) + W_o^l[\, u \odot \sigma(v)], \qquad [u; v] = \sum_{k=0}^{2} W_k^l\, h_i^{l-1}(t - kD_l), \quad D_l \in \{1, 3, 9, 27\} \tag{3}$$

T block: a causal, dilated, gated convolution along time at each node, with a residual connection. Four blocks give a receptive field of 1 + 2(1+3+9+27) = 81 steps (13.5 h) ≥ L. It lets each node extract trends, rates of rise and rain accumulations from its own history.

$$s_{ji}(t,\lambda) = v^\top \tanh(A\, z_j(t-\lambda) + B\, z_i(t) + C\, \phi(\Delta_{ji})) - \frac{(\lambda - \kappa_{ji}(t))^2}{2\varsigma^2}, \qquad \kappa_{ji}(t) = \frac{\Delta_{ji}}{c_{ji}(t)}, \quad c_{ji}(t) = \mathrm{softplus}(c_0 + \beta^\top z_j(t)) \tag{4}$$

$$\alpha_{ji}(t,\lambda) = \mathrm{softmax}_{\lambda \in \{0\,..\,\Lambda\}}\, s_{ji}(t,\lambda), \quad m_{ji}(t) = \sum_{\lambda=0}^{\Lambda} \alpha_{ji}(t,\lambda)\, W_m\, z_j(t-\lambda), \quad g_{ji}(t) = \sigma(w_g^\top [z_j(t); z_i(t)] + b_{ji}) \tag{5a}$$

$$h_i^l(t) = \mathrm{LN}(z_i^l(t) + W_2\, \mathrm{GELU}(W_1 z_i^l(t) + \sum_{j \in \mathcal{P}(i)} g_{ji}(t)\, m_{ji}(t))) \tag{5b}$$

S block (travel-time attention): node i looks at each upstream parent j at several earlier times t−λ and learns *which lag* to read, $\alpha_{ji}(t,\lambda)$. The Gaussian term is a physical prior centred on the expected travel time κ_{ji} = distance / celerity (in 10-min steps); celerity c_{ij} grows with the upstream state (β), so travel time shortens when the river is high (A6). Messages from all parents are *summed* (mass balance, A8). Lags satisfy λ ≥ 0, so nothing from the future of node j is used. On the future positions (t > τ) the released-water channel on Res carries the schedule, and the S block moves it downstream along the learned lags — this is how an announced release influences S1–S4 within the hour. After four blocks, S4 has read Res (4 hops), T1 (3), U (2) and S1–S3.

$$\hat{y}_i^{(q)}(\tau+h) = \tilde{w}_i(\tau) + \sigma_i\, \mathrm{MLP}_q([\, h_i^4(\tau+h);\, h_i^4(\tau)\,]), \quad h = 1..6,\ q \in \{0.1, 0.5, 0.9\} \tag{6}$$

Forecast mapping: persistence plus a learned change. $\tilde{w}_i(\tau)$ is the last valid level in metres (Eq. 1) and σ_i the standard deviation of one-hour level changes at that station. Quantiles are kept ordered by predicting the median and two non-negative offsets (softplus).

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \sum_{h=1}^{6} \sum_{i=1}^{4} \sum_{q} \omega_h\, \mu_{bih}^{y}\, \frac{\rho_q(y_{bih} - \hat{y}_{bih}^{(q)})}{\sigma_i} + \lambda_{\mathrm{wd}} \|\theta\|^2, \quad \rho_q(e) = \max\{qe, (q-1)e\}, \quad \omega_h = \frac{h}{21} \tag{7}$$

Training objective: pinball (quantile) loss, normalised by σ_i so the four stations count equally, masked by μ^y (missing targets), with larger weight on later horizons. AdamW, early stopping on validation MAE of the median at h = 6, chronological split (A1). Baselines for the committee: persistence and a linear ARX model on the same inputs.

How each source influences each forecast

Station	Upstream stations / tributaries	Rainfall	Reservoir release	Why this path
S1	none gauged upstream of the reservoir	R1 through r_{S1} (local runoff)	Res→S1: **actual** past release read at learned lag; **announced** release for τ…τ+60 enters directly if travel time < 1 h	The release is the main driver; without the schedule S1 is the hardest station because its water has not yet left the dam.
S2	S1 (1 hop), T1 (1 hop, added)	r_{S2}; rain also drives T1's future flow	2 hops (Res→S1→S2)	Water now at S1 and T1 arrives at S2 within the hour (A6, A9).
S3	S2 (1 hop), U (latent, 1 hop), T1 (2 hops)	drives U's inflow (A10); r_{S3}	3 hops	S2's current level carries most of the signal; U adds an unseen inflow.
S4	S3 (1 hop), U (2), S1–S2 (via S3), T1 (3)	indirect, through U, T1 and upstream	4 hops	The longest travel time: today's S3 and S2 levels are the best predictor.

Physical assumptions versus learned mappings

Physical assumption (built into the structure)	Learned mapping (fitted from data)
Directed downstream graph, no backwater (A7)	Stage–discharge relation at each station (inside the MLP and the T blocks)
Travel time = distance / celerity, prior in Eq. 4 and lag window Λ = 3 h (A6)	The celerity c_0, β, the spread of α (attenuation, dispersion) and each lag attention
Summed contributions at confluences; monotone, non-negative gates (A8)	Rainfall–runoff for T1, U and local runoff; gauge weights ω fine-tuned from the distance initialisation
Forecast = last level + change (persistence skip, Eq. 6)	The change itself; how reliable the announced schedule is (A12–A13); how much a gap in a series should be trusted (Eq. 1)
Future rain and levels are unknown, so they are masked placeholders (F6)	The unmonitored tributary U (latent, no direct supervision, A10)

Why this design. (1) A graph keeps physical locations: stations of unequal spacing and tributary junctions are explicit, instead of being forced into a regular grid or a flat vector. (2) Alternating T and S blocks (joint

space-time paths) lets a change at S1 be read at S2 at the right lag, which neither a temporal-only nor a spatial-only stack can do. (3) Lag attention is a small, interpretable way to learn changing travel times that were not supplied. (4) Persistence plus residual starts every forecast from the best cheap baseline. (5) Quantile outputs give a usable uncertainty without a separate probabilistic model. **Physical relationships and uncertainty (optional items).** No separate hydraulic simulator is run: physics enters as structure (directed graph, summed confluence inflows, travel-time prior, persistence skip) and as assumptions A6–A8, and every other relationship is learned. Uncertainty is represented by the 10/50/90 % quantile outputs (Eq. 6–7) and by flags that widen when inputs are stale or unannounced; it is not a full predictive distribution, and its calibration must be checked on held-out floods (U10).

5. Operation and exceptional conditions

The service loop at every tick τ: (1) freeze an *as-of snapshot* of the bitemporal archive (event time + arrival time) at τ + 2 min; (2) build X (Figure Q1.1, left half); (3) run the network; (4) store forecast, inputs' ages and flags; (5) publish.

Case	Handling	Reason / what the model sees
(i) Missing observations	Value not available at τ: m = 0, δ keeps growing, value filled by Eq. 1 (decay from last value to station mean). Training randomly masks 5–20 % of values, bursts up to 2 h, and sometimes a whole station for up to 24 h. If S_i is missing at τ the forecast for S_i is still produced from neighbours and flagged; intervals widen as δ grows.	The network was trained under the same gaps, so it learns to lean on neighbours and on the last valid value. Target gaps are masked in the loss.
(ii) Delayed observations	Availability is defined by *arrival* time: a record that arrives after τ + 2 min is treated as missing at τ (so it appears with age δ > 0). When it arrives later it is written into the archive and used in all following windows. Late-arriving corrections do not overwrite past forecasts; they are logged. Training uses delays sampled from the empirical arrival log (A3).	No leakage: the training windows are rebuilt as-of τ with simulated delays, so test-time staleness matches training.
(iii) Announced future releases	For each future step τ+10 … τ+60 take the most recent announcement made ≤ τ whose effective time covers that step; write q^{sch} on the Res time axis (zero-order hold) with $f^{sch} = 1$. Steps with no announcement get the current actual release and $f^{sch} = 0$. The model sees the schedule only on positions after τ; the S block carries it downstream with the learned lag.	Uses only information that exists at τ; the flag lets the model separate 'planned' from 'assumed unchanged'. Announcements made after τ are ignored by construction.
(iv) Scheduled versus actual release	Past steps use the metered *actual* release; future steps use the schedule. The recent deviation $e(t) = q^{act}(t) - q^{sch}(t)$ over the last 2 h is a feature on Res. Training pairs actual history with the schedule as announced at the time, so real mismatches (gate delay, emergency spill) are in the data; extra augmentation shifts schedule steps by 0–30 min and scales them by ±10 %. If |e| is large the 90 % quantile is automatically wider.	The model learns how much to trust a schedule from how well past schedules were followed (A13). An unannounced change is a limitation (§6).

Feasibility every 10 minutes. The network has ≈ 0.25 M parameters. One forecast reads N·T = 7·78 = 546 node-time positions: the temporal convolutions cost about 7·78·(3·64·128)·2 ≈ 27 MFLOP per block, and the lag attention (6 edges × 78 steps × 19 lags × 32 hidden units) is below 1 MFLOP per block once the projections are shared, so the whole forward pass is a few hundred MFLOP (my estimate; no benchmark was run) — well under a second on one CPU core, against a 600 s budget. Data access (a few kilobytes per tick) and publishing dominate. Training (≈ 263 000 windows × ~1 GFLOP including backward ≈ $3 \cdot 10^{14}$ FLOP per epoch) is offline, once per year or after a major flood. Fallbacks: if the model or a feed fails, publish the last-level persistence forecast with a 'degraded' flag; a health check compares each forecast with the level observed one hour later.

6. Uncertainty, limitations and additional information

ID	Uncertainty / limitation	Impact on the forecast	Response, or what remains unresolved
U1	Unmonitored tributary between S2 and S3	Unseen inflow changes S3 and S4 after rain; largest during storm onset	Latent node U driven by rain and S2→S3 mismatch; q0.9 widens at S3/S4. **Unresolved:** its true flow cannot be checked without a gauge.
U2	Travel times change with flow, season and gate operations	A wrong lag shifts the arrival of a wave and gives large errors on rising limbs	State-dependent lag attention (Eq. 4–5); retraining each year. **Remains:** extreme floods beyond the training range.
U3	No future rainfall (F6)	Rain falling in the next hour changes local and tributary levels that the model cannot see	Only past rain is used; the 90 % quantile covers part of it. **Unresolved** unless a rain nowcast is supplied (R2).
U4	Sparse rainfall sampling (two gauges)	Storm cells missed by R1/R2 make the runoff estimate wrong	Distance weights and uncertainty widening when T1/S rise without rain. **Unresolved**: needs more gauges (R4).
U5	Missing, late or faulty observations	Skill degrades with gap length δ	Masks, decay imputation, delay-aware training, flags (§5). Outages longer than ~24 h degrade to near-persistence.
U6	Unannounced or changed releases	S1 error grows when the dam acts differently from the schedule	Deviation feature and a wider q0.9. **Unresolved** for emergency spills without notice.

ID	Uncertainty / limitation	Impact on the forecast	Response, or what remains unresolved
U7	Rare, extreme floods and short record	The network may under-predict peaks outside the training range	Pinball loss with horizon weighting, evaluation on flood events separately, physical priors, persistence fallback. **Remains:** true extremes cannot be validated.
U8	Non-stationary channel (silting, vegetation, gauge drift)	Systematic bias and shifting stage–discharge	Season encoding, yearly retraining, drift monitoring against rolling error. **Remains** until a rating-curve update is supplied (R1).
U9	Backwater, abstractions, weirs not modelled (A7)	Local level offsets	Ignored (assumption A7). Documented; can be added as extra edges or inputs if information appears (R5).
U10	Calibration of quantiles	Intervals may be too narrow or too wide	Check coverage on validation per station and per flow class; recalibrate with conformal offsets. **Remains:** coverage in rare events is untested.

Additional information requests

ID	Request	How it would help	System usable without it?
R1	Stage–discharge (rating) curves and cross-sections for S1–S4, T1	Converts levels to discharge so mass balance at confluences can be checked and applied; reduces U8	Yes. The curves are learned from data.
R2	Rainfall nowcast or radar for the next 1–3 h	Gives the model the future rain it lacks (U3); biggest gain at storm onset	Yes. The mask handles absence and the 90 % quantile covers part of the risk.
R3	A gauge or even a temporary stage logger on the unmonitored tributary	Turns U from latent into observed (U1); direct supervision of its rain response	Yes. It remains a latent node.
R4	More rain gauges in the catchment	Better areal rainfall and fewer missed cells (U4)	Yes. The distance weights adapt to what exists.
R5	Reservoir gate positions, reservoir level, operating rules, any abstractions or weirs	Explains release deviations (U6) and local offsets (U9); better release model for the Res node	Yes. The system relies on actual and scheduled flow.
R6	History of past flood events with at least 5–10 years of records, and sensor QC flags and measurement timestamps	More extreme events for training and honest evaluation (U7); cleaner masks (U5)	Yes. With fewer years use a smaller model and rely on priors.
R7	Reporting-delay log (arrival times) and release announcement log	Exact as-of reconstruction for training (§5 ii, iii)	Yes, with simulated delays as a substitute (A3, A12).
R8	Flood-stage thresholds at S1–S4	Evaluate alert skill (hit rate, false alarms) and tune the quantile that triggers warnings	Yes. Forecasts remain valid; alert tuning would be generic.

Question 2 — Motion or Memory?

Important notes on how the experiments were run (read first)

- **Reduced training budget.** The handout asks for 30 epochs per model. All numbers below come from **6 epochs per model** (1 500 Adam steps, the same for A, B, C and D and for both seeds), because the only machine available was a CPU laptop (AMD Ryzen 5 5500U, 6 cores, no GPU, PyTorch 2.10 CPU). One epoch of model A took about 11 minutes there; the full protocol (8 runs × 30 epochs) would need roughly 20–25 hours. Everything else follows the handout (Adam 1e-3, batch 32, one random window per video per epoch, per-video masked loss, no teacher forcing, final-epoch weights, seeds 0 and 1). **As a result the models are under-trained** (they predict soft blobs, not sharp digits, Figure 2), so absolute errors are larger than a 30-epoch model would give, and some conclusions below may change at 30 epochs. The same code runs the full protocol with `EPOCHS=30 bash run_all.sh` (it also uses a GPU if one is present).
- **Timing.** Runs were executed 2–4 at a time, which distorts wall-clock time, so Tables 5 and 6 report training time per epoch and inference time measured *afterwards, alone on the machine* (training: mean of 12 real training steps × 250 steps per epoch; inference: all 1 000 test videos, batch 32, `eval()` and `no_grad()`). The mean epoch times logged during the concurrent runs are given under Table 5.
- **Optional model D+B** was not trained (compute). The final model is therefore the symmetrised model D (Section 5).
- **Model D's design** was screened on the validation split only, with 250 training steps per candidate; both candidates were still at the 'static blob' plateau at that point, so the screening is almost a tie (Q2.12).
- **Q2.5 and Q2.11** are quoted exactly as committed to the repository before the corresponding experiments (commit times 12:17 and 12:22 on 25 October 2026, file `report/q2_predictions.md`). Where they turned out wrong the answers that follow say so.
- All performance numbers use the 1 000 test videos (dataset indices 9000–9999); the validation split was used only for implementation checks and for model D's screening. Table entries are mean (seed 0, seed 1); figures and GIFs use seed 0.

1. Setup

Table 1. Checks of the transformation and window functions (the same functions used in all experiments; synthetic tensors).

Check	Expected	Yours
(a) max|M(M(x)) - x|	0	0, torch.equal=True
(b) max|R^4(x) - x|	0	0, torch.equal=True
(c) max|M R M(x) - R^-1(x)|	0	0, torch.equal=True
(d) bright pixel of M(p) (frame, row, column)	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of R(p) (frame, row, column)	(7, 43, 10)	(7, 43, 10)
(f) 1x input / target frames	1-5 / 6-10	1-5 / 6-10
(g) 2x input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h) 3x input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19
(extra) g^-1(g(x)) == x for all 8 g in G; number of distinct group elements	True; 8	True; 8

All rows match. (The extra row also confirms that the eight elements of G are distinct and that $g^{-1}(g(x)) = x$ for each of them.)

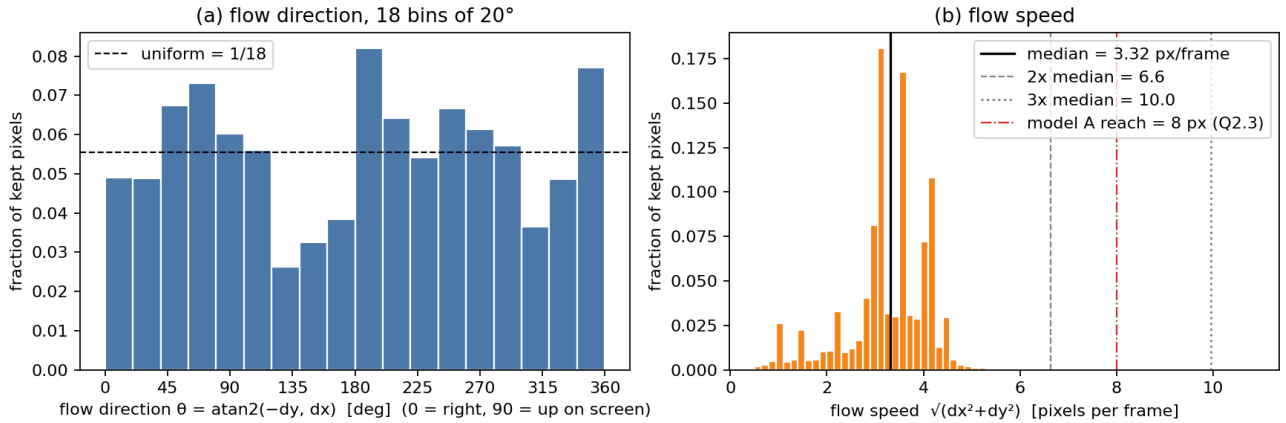


Figure 1. Optical flow (OpenCV Farneback, the exact settings of the handout) on the first 50 training videos, 950 frame pairs. Pooled kept pixels (intensity > 0.2 in both frames, flow magnitude > 0.5 px): **103,050**; median speed **3.32 pixels per frame**. (a) direction θ with the uniform level 1/18; (b) speed, with the median, 2× and 3× the median, and model A's reach (Q2.3) marked.

Q2.1

Directions. Mirroring maps θ to 180° − θ and a 90° anticlockwise rotation maps θ to θ + 90°; a histogram that is flat is unchanged by both. Figure 1(a) is roughly flat but lumpy: bins run from 0.026 to 0.082 against 1/18 = 0.056. With only 50 videos (about 100 digit trajectories) this is mostly sampling noise: the total-variation distance between the histogram and its mirrored version is 0.16 and for the rotated version 0.21 (10° bins), while two independent halves of the same data (25 versus 25 videos) are already 0.18 apart. As a check beyond Figure 1, on 1 000 videos the histogram is much flatter (bins 0.040–0.069, distance to uniform 0.07). So **mirrored and rotated trajectories are statistically familiar** to a model trained on this data; the training data is (nearly) symmetric in *motion*.

Speeds. The typical speed is the median 3.32 px/frame (90th percentile 4.1; on 1 000 videos the 99th percentile is 4.6). Speed-ups are therefore 2× ≈ 6.6 and 3× ≈ 10.0 px per frame, well outside anything in the training videos, so fast motion is **unfamiliar**, unlike mirror and rotation. **Overlaps.** Farneback is unreliable where the two digits overlap, but those pixels are a small share of the data and are partly removed by the intensity and magnitude filters. Dropping every frame pair in which the thresholded digits merge into one blob (leaving 489 of 950 pairs) changes the median speed only from 3.32 to 3.26 and the mirror and rotation distances from 0.16/0.21 to 0.19/0.23, within the noise level above. The overlap bias is not large enough to change the answer.

Q2.2

Mirror: changes the *appearance* (a mirrored 2 is not a 2) but, given Q2.1, not the motion statistics. **Rotation by 90°**: also changes only appearance, and more strongly (strokes that were vertical become horizontal; the patch layout of the network no longer matches), while motion statistics stay the same. **Speed-up**: changes the *motion* statistics (speeds the model never saw) and leaves digit appearance unchanged. Hence the three transformations form a controlled experiment: mirror and rotation vary appearance with motion held (statistically) fixed, speed varies motion with appearance held fixed, so a failure can be assigned to one cause.

2. Baseline: ConvLSTM

Model A was implemented exactly as specified (patch folding with `pixel_unshuffle(x, 4)`, two ConvLSTM layers with 64 channels and 3×3 kernels, one convolution producing [i, f, o, g], a 1×1 output convolution, `pixel_shuffle`, sigmoid; 9-step rollout feeding back its own predictions, steps 5–9 are the forecast). It has **480 784** trainable parameters, as required.

Q2.3

One cell of the folded 16×16 grid is 4 original pixels, so each 3×3 convolution moves information by at most one cell = **4 px** (centre to centre). Within one time step τ there are three kinds of path to the prediction $\hat{x}_{\tau+1}$: (a) new input → layer-1 convolution (4 px) → $h^{(1)}_{\tau}$ → layer-2 convolution on its input part (4 px) → 1×1 output (0 px): **8 px**; (b) previous layer-1 state $h^{(1)}_{\tau-1}$ → layer-1 convolution on its recurrent part (4 px) → layer-2 input convolution (4 px): **8 px**; (c) previous layer-2 state $h^{(2)}_{\tau-1}$ → layer-2 recurrent convolution (4 px) → output: 4 px. The longest path is two convolutions, so **the reach of model A is 2 × 4 = 8 original pixels per time step** (the 1×1 convolution, pixel_unshuffle and pixel_shuffle add nothing). Compared with Figure 1(b): normal motion (median 3.3 px) is well inside 8 px; 2× (6.6 px, fastest digits ≈ 9 px) is near the limit; 3× (10 px) is beyond it.

Table 2. Per-horizon test MSE (h = forecast horizon), $E^{(5)}$ and trainable parameters at 1× with t = 1. Model A: mean (seed 0, seed 1).

Predictor	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	$E^{(5)}$	Trainable params
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0
Model A (mean (seed 0, seed 1))	0.0338 (0.0294, 0.0382)	0.0347 (0.0309, 0.0384)	0.0357 (0.0328, 0.0387)	0.0367 (0.0345, 0.0389)	0.0376 (0.0360, 0.0391)	0.0357 (0.0327, 0.0387)	480,784

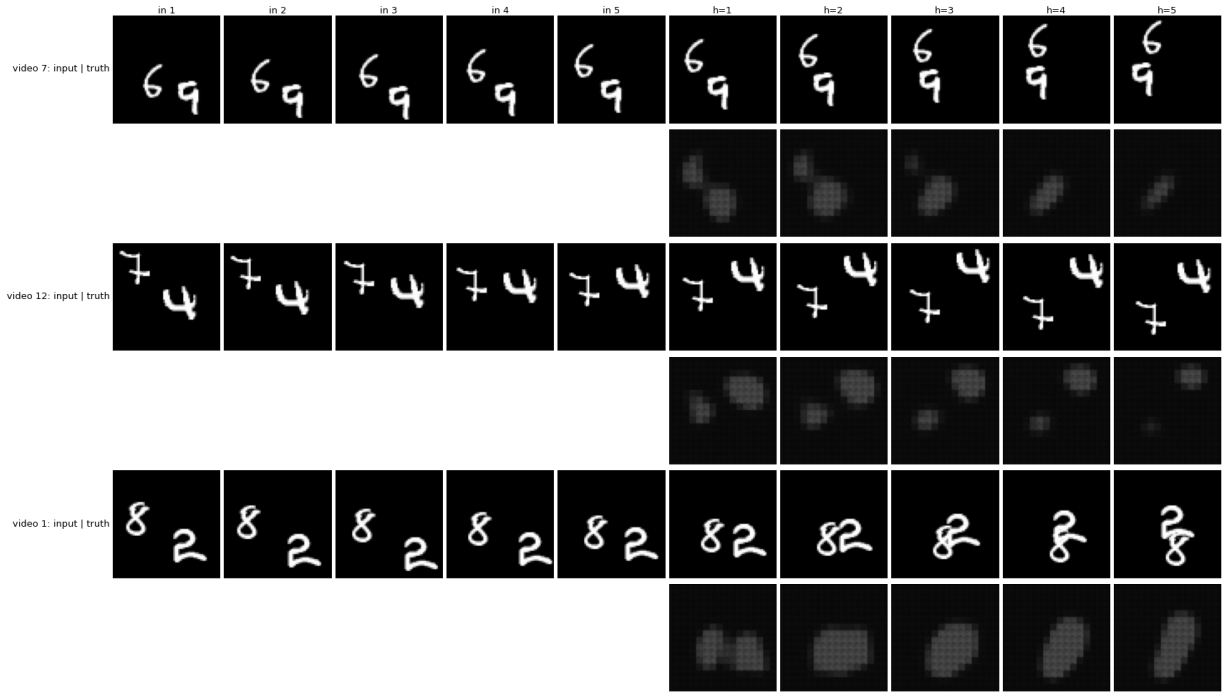


Figure 2. Model A (seed 0): inputs and true future frames (first row of each video) and the prediction (second row), for three test videos picked by a rule that uses only the ground truth: videos 7 and 12 are the first with two separate digits in all ten frames; video 1 is the first whose two digits are clean in the inputs and merge into one blob in at least two target frames (overlap).

Q2.4

Model A is better than both reference predictors in MSE: $E^{(5)}$ = 0.0327 (seed 0) and 0.0387 (seed 1), against 0.0426 for Blank and 0.0641 for Copy-last. Copy-last is worse than Blank because a copied digit in the wrong place is penalised twice (where it was and where it should be). **Error growth:** Blank is flat. Copy-last grows by 49% from h = 1 to h = 5 (0.0492 to 0.0732), because digits keep moving. Model A seed 0 grows by 22% (0.0294 to 0.0360), a slower but steady rise. Seed 1 is almost flat (0.0382 to 0.0391): it has found only a static blur and uses almost no motion information (large seed difference at this training length). **Figure 2** shows what 'better than Blank' means: A predicts soft grey blobs at about the right places that drift in the right direction and fade with the horizon, not sharp digits; blur lowers MSE but the digit identity is lost. **Overlap** (video 1): as the two digits approach each other the two blobs merge into one elongated blob and A has no way to say which digit continues where; the true digits pass through each other and re-appear.

Q2.5 (quoted exactly as committed before the experiments of Section 3)

Mirror: I expect model A to mostly pass. Figure 1(a) shows directions spread over all angles (no strongly preferred direction), so mirrored trajectories are familiar, and Figure 2 shows that A predicts soft blobs rather than sharp digits, so it should rely little on what a digit looks like; I expect a small consistency gap C_M and E_M close to E(5) on the original clips. Rotation: also mostly pass, but slightly worse than mirror (C_R >= C_M and E_R >= E_M) because rotated digits look even less familiar and the 4x4 patch folding and fixed kernels are not rotation-symmetric; the direction of motion is still familiar from Figure 1(a). Speed: my reach estimate for A is 8 original pixels per step (Q2.3) and the median training speed is 3.3 px per frame (Figure 1b). So 2x (about 6.6 px per frame, the fastest digits about 9 px) should still be tracked but with a visible loss of accuracy, and 3x (about 10 px, the fastest about 14 px) is beyond the reach, so I expect A to break down at 3x: digits stall or vanish, and E(2) at 3x should be at least as bad as Copy-last.

Outcome: mirror and rotation passed as predicted (small gaps, Table 3). The speed prediction was only partly right: A does lose accuracy with speed, but at 3× it is not 'as bad as Copy-last' because blurred predictions are rewarded by MSE (Table 4); it is almost as bad as Blank. See Q2.9.

3. Testing the Baseline

Table 3. $E^{(5)}$ and the consistency gap C and transformed error E under mirror (M) and rotation (R), 1× and t = 1 (mean (seed 0, seed 1)). Blank and Copy-last are deterministic.

Predictor	$E^{(5)}$	C_M	E_M	C_R	E_R
Model A (mean (s0, s1))	0.0357 (0.0327, 0.0387)	1.90e-04 (2.33e-04, 1.46e-04)	0.0357 (0.0327, 0.0387)	2.71e-04 (4.03e-04, 1.39e-04)	0.0358 (0.0329, 0.0387)

Predictor	$E^{(5)}$	C_M	E_M	C_R	E_R
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

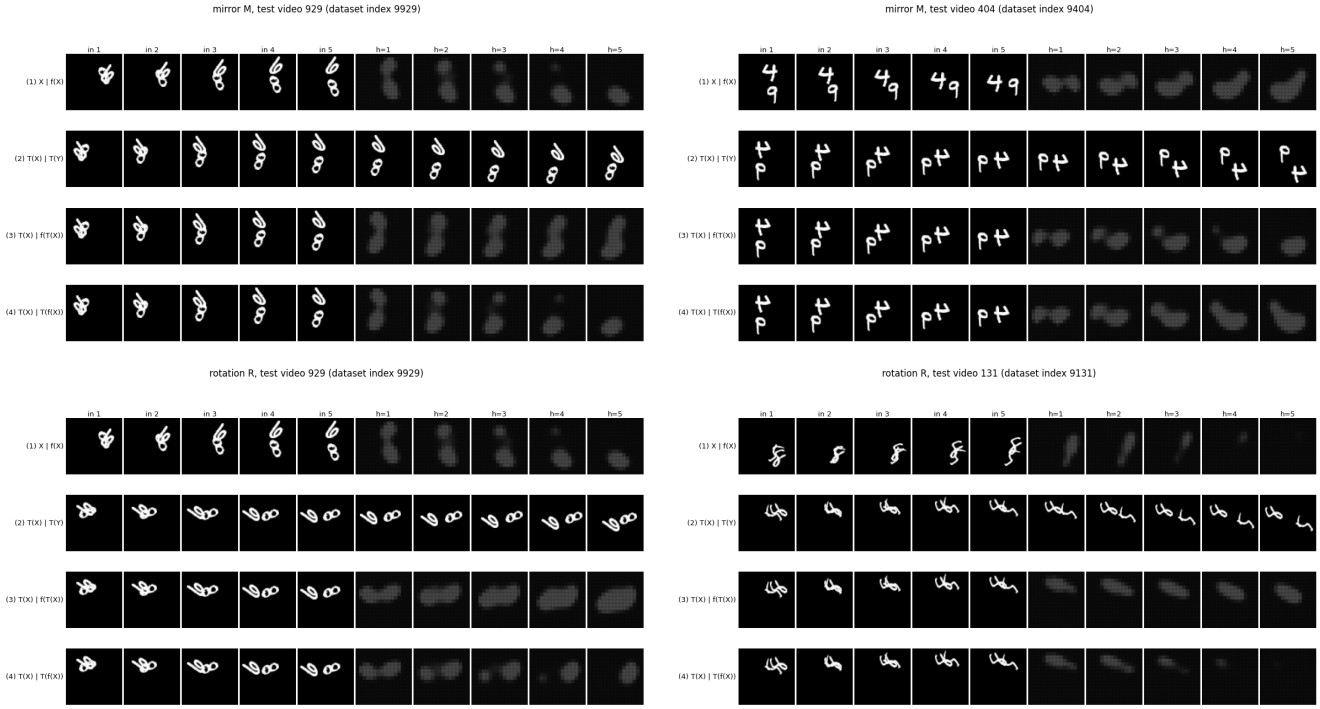


Figure 3. Model A (seed 0) on the two test videos with the largest per-video C_M (top row: videos 929 and 404) and the two with the largest C_R (bottom row: videos 929 and 131); video 929 is among the top two for both, so it appears twice. Rows: (1) X | f(X); (2) T(X) | T(Y); (3) T(X) | f(T(X)); (4) T(X) | T(f(X)); columns 1–5 inputs, columns 6–10 forecasts h = 1…5. Animated GIFs (the same four rows stepping through h = 1…5): `Question2/gifs/fig3_A_M_test929.gif`, `Question2/gifs/fig3_A_M_test404.gif`, `Question2/gifs/fig3_A_R_test929.gif`, `Question2/gifs/fig3_A_R_test131.gif`.

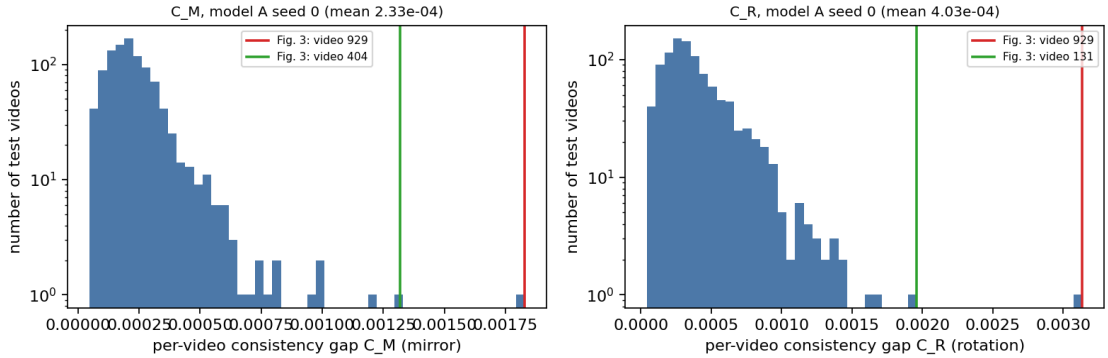


Figure 4. Per-video C_M and C_R of model A (seed 0) over the 1 000 test videos (log counts); the videos of Figure 3 are marked.

Q2.6

Blank and Copy-last have C = 0 *exactly* (a zero map and a copy map commute with any pixel permutation) but $E_M = E_R = E^{(5)} = 0.0426$ and 0.0641: perfectly consistent and useless. So C measures only whether a model treats the transformed scene like the original, not whether it is right; a constant or blurry output can be made consistent at no cost. E_T must therefore always be reported next to C_T. Model A is the same case in milder form: C_M and C_R (2.33e-04 and 4.03e-04 for seed 0) are about 100× smaller than E (0.0327, 0.0329) and E_M, E_R are equal to $E^{(5)} = 0.0327$ to three digits: the model is not hurt by mirroring or rotating, but it is also not accurate enough on any of them to expose a failure.

Q2.7

Video 929, mirror (Figure 3, top-left). Rows 3 and 4 differ (MSE between them 0.00183, about 5% of the error against the truth, 0.0383), but in a clear way. The truth (row 2) is two digits sliding down, with the same ink mass in all frames. In row 4, T(f(X)), the model has the digits in separate blobs: the upper blob fades and the lower one moves down; its total brightness falls from 314 to 260 (truth 262) and its centre of mass moves 6.6 px down (truth 8.7). In row 3, f(T(X)), the two digits blur into **one tall blob that barely moves and grows**: brightness rises from 322 to 363 and the centre of mass moves only 3.7 px. So when it sees the mirrored scene the model's account changes from 'two digits, one leaving' to 'one smeared object that stalls', even though the

true motion is the mirror image of the first scene. (Both rows are wrong against the truth, 0.0383 and 0.0414.)

Q2.8

Slightly, for seed 0; not for seed 1. Seed 0: C_R = 4.03e-04 is 1.73× C_M = 2.33e-04, and C_R is larger than C_M on 90% of the test videos (median ratio 1.60); E_R = 0.0329 is marginally above E_M = 0.0327. Seed 1 shows no difference (C_R = 1.39e-04, C_M = 1.46e-04), as expected for a nearly static predictor. By Q2.1 the *motion* statistics are the same for both transformations (flat direction histogram), so the motion cannot explain the difference. By Q2.2, rotation changes the *appearance* more than mirroring (stroke orientation, the patch layout), so the extra gap for seed 0 is attributed to appearance. The size of the effect is small relative to the error, and it rests on one informative seed of an under-trained model.

Table 4. Faster motion, t = 1: $E^{(2)}$ (first two horizons only, for comparability across speeds) at 1×, 2×, 3× and $E^{(5)}$ at 1× and 2×.

Predictor	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	$E^{(5)}$ 1×	$E^{(5)}$ 2×
Model A (mean (s0, s1))	0.0342 (0.0302, 0.0383)	0.0376 (0.0363, 0.0390)	0.0397 (0.0400, 0.0394)	0.0357 (0.0327, 0.0387)	0.0394 (0.0395, 0.0394)
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719
Blank (reference)	0.0426	0.0425	0.0426	0.0426	0.0425

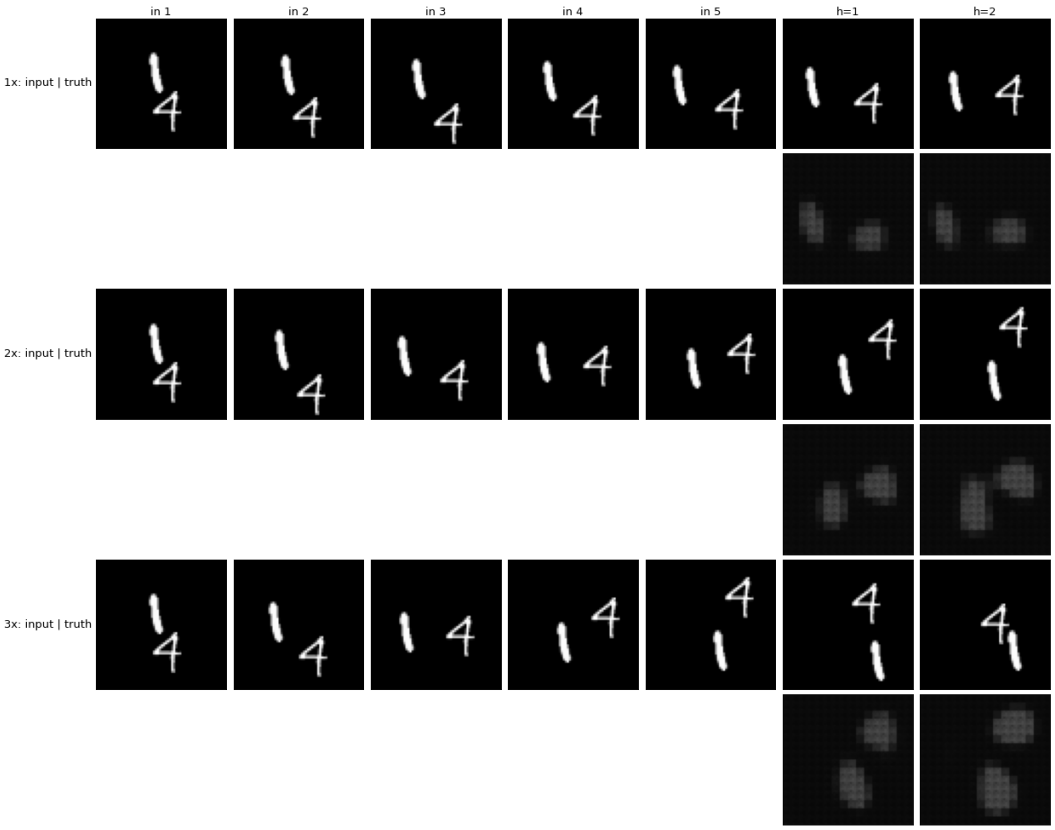


Figure 5. Test video 21 (digits do not overlap in any displayed true frame; the selection used only the ground truth). Model A (seed 0): five inputs, true h = 1, 2 (first row of each speed) and predicted h = 1, 2 (second row) at 1×, 2× and 3×.

Q2.9

Where it falls apart. $E^{(2)}$ of model A (seed 0) rises from 0.0302 at 1× to 0.0363 at 2× (+20%) and 0.0400 at 3× (+33%), which is 94% of the Blank error 0.0426: at 3× the forecast has almost no skill left, at 2× it is clearly degraded already. (Seed 1, the near-static predictor, is already at 0.0383 at 1× and stays there.) **Against the reach estimate (8 px, Q2.3):** reach predicted trouble only beyond about 2.4× the typical speed, i.e. at 3×. The data say trouble starts already at 2×. What my estimate missed is that reach limits how far a single update can connect old and new positions, but the model also has to *measure* speed from the inputs and use it. The measurement shows it does not: the correlation of its h = 1 prediction with the last input frame stays at 0.91, 0.92 and 0.90 at 1×, 2×, 3×, while its correlation with the true frame falls from 0.92 to 0.70 to 0.49 (copying the last input would give 0.87, 0.64, 0.43). (The digits are about 20 px wide, so a digit that jumps 7–10 px still overlaps its old position and a convolutional state can partly follow it, which is another reason why reach is a soft limit.) **What the failure looks like (Figure 5):** grey blur and a **stall**. At 1× the two blobs sit close to the true digits. At 2× and 3× they are still centred near where the digits were in the last input and move only a few

pixels, while the true digits have jumped ~7–10 px; there is no duplicated digit and no vanishing digit in this video, only blobs that lag behind. The blobs' total brightness is 1.5× the true ink mass at 3× (it is 1.4× at 1×): the model spreads its uncertainty. A stall suggests the model is applying the typical 1× displacement it learned (≈ 3 px per frame) instead of the observed one: it behaves like a *memorised speed prior* plus blur, not a measured velocity.

Q2.10

No. The two explanations make the same prediction for A: unfamiliar speeds and a limited reach both make fast clips fail, and A was trained only at 1× with an 8 px reach, so both are present at once. An experiment that separates them changes only one of them: (i) train the *same* architecture (same 8 px reach) on clips at 1×, 2× and 3× (this is model C): if it then handles 3× well, the failure was unfamiliar motion; if 3× stays bad, reach (or capacity) is the limit. (ii) Train a model with a larger reach on 1× clips only and test it at 2× and 3×: if it improves although it never saw fast motion, reach was the bottleneck. Comparing A, C and the larger-reach model D gives the answer (Section 4).

4. What Can Data Fix?

Q2.11 (quoted exactly as committed before B, C and D were trained)

B (symmetry augmentation) should change little: model A already shows small C_M and C_R and E_M, E_R close to E(5), so there is no large mirror or rotation failure for it to fix, and the speed failure is untouched. C (speed augmentation) should fix most of the 2x failure, because those motions are within A's 8-pixel reach but were never seen in training, and only part of the 3x failure, since 3x motion (about 10 px per frame) is beyond A's reach. D (larger reach plus speed augmentation) should fix both 2x and 3x, with little or no change at 1x.

Table 5. Models A–D (6 epochs each; mean (seed 0, seed 1)). $E^{(5)}$, C and E at 1×, t = 1; $E^{(2)}$ at 1×, 2×, 3×. Training time per epoch (250 steps) and inference time (1 000 test videos, batch 32) were measured alone on the CPU after training (AMD Ryzen 5 5500U, 6 threads).

Model	$E^{(5)}$ 1×	C_M	E_M	C_R	E_R	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Params	Train s / epoch	Inference s
A	0.0357 (0.0327, 0.0387)	1.90e-04 (2.33e-04, 1.46e-04)	0.0357 (0.0327, 0.0387)	2.71e-04 (4.03e-04, 1.39e-04)	0.0358 (0.0329, 0.0387)	0.0342 (0.0302, 0.0383)	0.0376 (0.0363, 0.0390)	0.0397 (0.0400, 0.0394)	480,784	436 (436, 437)	20.1 (20.4, 19.8)
B	0.0358 (0.0327, 0.0389)	2.43e-04 (3.15e-04, 1.71e-04)	0.0358 (0.0327, 0.0389)	2.18e-04 (2.78e-04, 1.57e-04)	0.0358 (0.0327, 0.0389)	0.0344 (0.0302, 0.0386)	0.0377 (0.0363, 0.0391)	0.0399 (0.0402, 0.0395)	480,784	439 (437, 441)	20.2 (20.4, 20.1)
C	0.0361 (0.0355, 0.0366)	1.82e-04 (1.08e-04, 2.57e-04)	0.0361 (0.0355, 0.0366)	2.54e-04 (1.46e-04, 3.62e-04)	0.0361 (0.0355, 0.0366)	0.0347 (0.0338, 0.0357)	0.0360 (0.0353, 0.0367)	0.0377 (0.0373, 0.0380)	480,784	436 (434, 438)	20.2 (19.9, 20.6)
D	0.0376 (0.0393, 0.0360)	3.96e-05 (2.70e-05, 5.22e-05)	0.0376 (0.0393, 0.0360)	4.22e-05 (3.21e-05, 5.23e-05)	0.0376 (0.0393, 0.0360)	0.0372 (0.0393, 0.0351)	0.0375 (0.0392, 0.0359)	0.0382 (0.0393, 0.0371)	594,496	144 (143, 145)	6.7 (6.8, 6.6)

Times logged *during* training are not comparable: A ran with one other job (661 s and 662 s per epoch); B, C, D ran with up to three other jobs, and the laptop went to sleep once, which inflated some epochs to hours. Model D (patch 8) is about 3.1× cheaper per epoch than A.

Q2.12

Model D = model A with patch size 8: `pixel_unshuffle(x, 8)` gives 64 × 8 × 8; two ConvLSTM layers with 64 hidden channels and 3×3 kernels; a 1×1 convolution 64→64; `pixel_shuffle(·, 8)` and sigmoid; same 9-step rollout without teacher forcing, same training. It has **594 496** parameters (≤ 721 176; 1.24× model A). No attention. **Reach** (same conventions as Q2.3): one cell is now 8 px. (a) new input → layer-1 conv (8 px) → layer-2 conv (8 px) → 1×1 output: 16 px; (b) previous layer-1 state → layer-1 conv (8) → layer-2 conv (8): 16 px; (c) previous layer-2 state → layer-2 conv (8): 8 px. Longest path = 2 × 8 = **16 px per step, strictly larger than A's 8 px**. It covers even the fastest 3× digits (99th percentile of training speed 4.6 px/frame × 3 ≈ 14 px). **Design choice on validation only:** two candidates with reach 16 px were trained for 250 steps (speed-augmented, seed 0) and scored on the validation split by $E^{(5)}$@1× + $E^{(2)}$@2× + $E^{(2)}$@3×: patch 8 with 3×3 kernels (594 496 params) 0.1181; patch 4 with 5×5 kernels and 32 hidden channels (359 184 params) 0.1193. Both were still at the static-blob plateau (E ≈ 0.039–0.040 at every speed), so this is almost a tie; I took the lower score, which is also the cheaper model per step. The consequence is visible below: model D is coarse (8×8 grid) and learned more slowly in 6 epochs.

Q2.13

Caveat first: after 6 epochs seed 1 of A and B and seed 0 of D are close to a static blur (E(2) ≈ 0.0383–0.0393 at every speed), so only some seeds carry information. Labels:

- **Mirror and rotation failure → none to label, and in particular not 'appearance'.** For A, C_M = 2.33e-04, C_R = 4.03e-04 are about 1% of E; model B (symmetry augmentation) does not reduce them (seed 0: 3.15e-04 and 2.78e-04), and E_M, E_R equal $E^{(5)}$ for A, B, C and D. At this training length there is no appearance-driven failure to remove (Table 5, columns C_M … E_R).

- **Loss of accuracy with speed → partly unfamiliar motion.** Speed augmentation (A→C) lowers $E^{(2)}$ at 3× from 0.0400 to 0.0373 (seed 0), and at 2× from 0.0363 to 0.0353; the gap between 1× and 3× shrinks from 0.00982 (A) to 0.00358 (C seed 0) and 0.00226 (C seed 1). The correlation of the h = 1 prediction with the true frame at 3× rises from 0.49 (A) to 0.61 (C seed 0) and 0.59 (C seed 1). The speed prior learned from 1× data was a large part of the failure.
- **Remaining 3× gap → architectural reach cannot be confirmed.** If C were still bad at 3× while D were good, the failure would be reach. Here D is *not* clearly better than C: at 3× D seed 1 gives 0.0371 against 0.0373 and 0.0380 for C, and D seed 0 never left the static blur (0.0393). With 6 epochs I can neither show nor rule out a reach effect; D's coarser grid and slower learning confound the comparison.

Why more data cannot carry information further than the architecture allows. Training data only changes the weights; the set of pixels that can influence an output in one update is fixed by kernel size, patch size and depth. If a digit moves farther than the reach, the old and new positions never meet inside one convolution, whatever the weights are, so no amount of fast training clips can create a path between them. Data can teach the network to *use* the available reach (what C does for speeds within it), but only a larger reach (D) can extend it.

Q2.14

Model C vs A at 1×: mean $E^{(5)}$ is 0.0361 against 0.0357; seed 0 is worse (0.0355 vs 0.0327, +9%) and seed 1 better (0.0366 vs 0.0387, because A seed 1 was stuck). So there is a small cost for the informative seed, in return for a visibly better 2× and 3× (Q2.13): I judge it worth it. **Model D vs A:** mean $E^{(5)}$ 0.0376 (seed 0 0.0393, seed 1 0.0360), about 5% worse than A, and one seed collapsed; the larger reach did not pay off in 6 epochs, so for this budget D was not worth it (it is, however, 3× cheaper to train). **B vs A:** no change (0.0358 vs 0.0357).

5. The Final Model

Why $\bar{f}$ has $C_M = C_R = 0$ for any f. Let $h \in G$. Each $g \in G$ is a pixel permutation (linear). $\bar{f}(h(X)) = \frac{1}{8} \Sigma_{g \in G} g^{-1}(f(g(h(X))))$. Put $g' = g \circ h$; as g runs over G so does g' (G is a group), and $g^{-1} = h \circ g'^{-1}$. Then $\bar{f}(h(X)) = \frac{1}{8} \Sigma_{g'} h(g'^{-1}(f(g'(X)))) = h(\bar{f}(X))$, since h is linear and commutes with the average. Taking h = M and h = R (both in G) gives $\bar{f}(T(X)) = T(\bar{f}(X))$, i.e. $C_M = C_R = 0$, whatever f is. In floating point the sum is evaluated in a different order for the two sides, hence values of order 10^{-17} instead of 0.

Choice of final model. Symmetrised model D (as the handout's default). D+B was not trained (optional, no compute left), and Table 5 gives no reason for it: B did not improve consistency or accuracy over A. **Caution:** D seed 0 never left the static blur, so the seed-0 figures below show an almost constant forecast; the seed-1 model does learn some motion.

Table 6. Table 5 columns for symmetrised B and symmetrised D (the final model). The training-time column is the training time of the underlying model (B or D). C_M and C_R in scientific notation.

Model	$E^{(5)}$ 1×	C_M	E_M	C_R	E_R	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Params	Train s / epoch	Inference s
sym B	0.0357 (0.0326, 0.0388)	2.78e-17 (3.97e-17, 1.59e-17)	0.0357 (0.0326, 0.0388)	2.22e-17 (3.19e-17, 1.26e-17)	0.0357 (0.0326, 0.0388)	0.0344 (0.0302, 0.0386)	0.0377 (0.0363, 0.0390)	0.0398 (0.0401, 0.0394)	480,784	439 (437, 441)	165.2 (164.7, 165.7)
sym D (final)	0.0376 (0.0392, 0.0360)	1.93e-17 (1.39e-17, 2.46e-17)	0.0376 (0.0392, 0.0360)	1.76e-17 (1.34e-17, 2.19e-17)	0.0376 (0.0392, 0.0360)	0.0372 (0.0393, 0.0351)	0.0375 (0.0392, 0.0359)	0.0382 (0.0393, 0.0371)	594,496	144 (143, 145)	54.2 (53.2, 55.2)

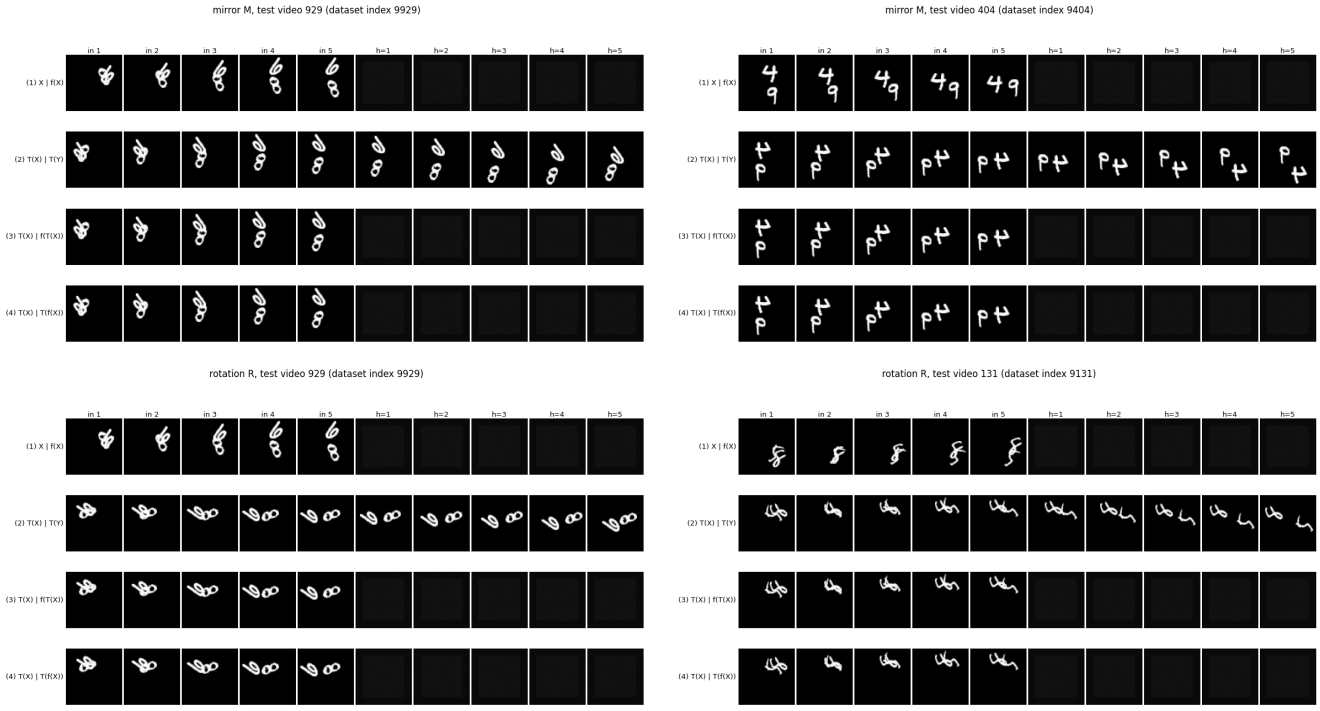


Figure 6(a). Figure 3 repeated with the final model (symmetrised D, seed 0) on the same four panels. Rows 3 and 4 are identical up to 10^{-17}. GIFs: `Question2/gifs/fig6a_Dsym_M_test929.gif`, `Question2/gifs/fig6a_Dsym_M_test404.gif`, `Question2/gifs/fig6a_Dsym_R_test929.gif`, `Question2/gifs/fig6a_Dsym_R_test131.gif`.

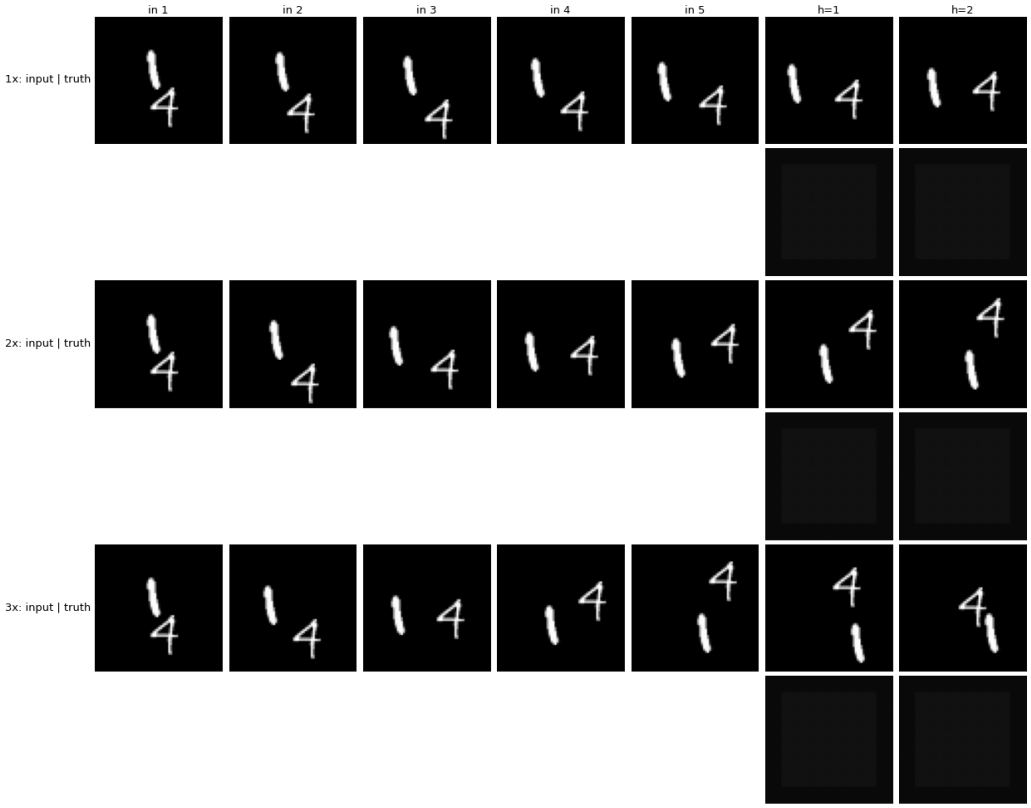


Figure 6(b). Figure 5 repeated with the final model (symmetrised D, seed 0) on test video 21: forecasts at h = 1, 2 for 1×, 2×, 3×.

Q2.15

Consistency. Learned symmetry (B) is approximate: C_M = 3.15e-04, 1.71e-04 and C_R = 2.78e-04, 1.57e-04 for seeds 0 and 1, no smaller than the un-augmented A (2.33e-04, 4.03e-04). Built-in symmetry is exact: C_M = 3.97e-17 and C_R = 3.19e-17 (sym B seed 0); 1.39e-17 and 1.34e-17 (sym D seed 0), zero up to floating point, because the average over a group is invariant under the group by construction (proof above), for any f.
Accuracy. On original clips symmetrisation changes $E^{(5)}$ very little: B seed 0 0.0327 → 0.0326, seed 1 0.0389 →

0.0388; D seed 1 0.0360 → 0.0360. On transformed clips E_M and E_R equal $E^{(5)}$ for every model, so there is nothing to gain or lose there either. **Cost.** Inference needs 8 forward passes: 20.4 s → 164.7 s for B (8.1×) and 6.8 s → 53.2 s for D; training cost is unchanged, whereas learned symmetry (B) costs nothing at inference but gives no guarantee. **Blur.** Averaging eight predictions hurts when the base model's predictions are sharp but not equivariant: the average of eight slightly displaced sharp digits is blurred and loses accuracy. It did not hurt here, because these under-trained models already predict blur, and the averaging even lowered the error slightly.

Q2.16

No. Video 929 (Q2.7) now gives identical rows 3 and 4 (Figure 6a), so the *inconsistency* is fixed by symmetrisation. But the final model (D seed 0) outputs an almost constant dim image at every horizon and does not move any digit, so the stalled, smeared account is not replaced by a correct one ($E^{(5)} = 0.0392$ for the whole test set). Symmetrisation fixed the disagreement between rows 3 and 4; it cannot fix a base model that has not learned the motion, and the larger reach of D and the speed augmentation did not fix that within 6 epochs.

Q2.17 Final verdict

No: this final model has not learned motion. Its seed-0 forecasts are almost constant, so zero consistency ($C = 10^{-17}$) is the trivial kind seen with Blank (Q2.6), and E is near the Blank error. A skeptic would want: sharp, correctly moved digits at 1× that beat Copy-last and Blank by a wide margin on both seeds; the same quality after mirror, rotation and 2×/3× speed-up (E_T and $E^{(2)}$ close to the 1× values); and correct behaviour at wall bounces and digit overlaps. The test it has clearly not passed is accuracy on the original clips itself, and with it the speed test, since $E^{(2)}$ at 3× is no better than at 1×. A 30-epoch run is needed before any claim about motion.

AI-use disclosure

Tool used: Claude Code (Anthropic), model Claude Sonnet 5.5, run inside the project folder. Below are the prompt given to the tool, a description of what it produced, and the edits made. **Student note:** the student must read this section and the whole report, then add their own edits in the last row before submitting.

Prompt (verbatim, as given)

First, inspect every file I uploaded carefully. There is a PDF that contains the complete assignment instructions, requirements, steps, and submission details. Do not start solving immediately. First, study the assignment completely and understand: what the assignment is asking me to do; every requirement and instruction in the PDF; what files, code, data, or other materials are provided; which files I need to use; what steps need to be completed and in what order; what calculations, analysis, coding, screenshots, outputs, explanations, or evidence are required; any formatting, naming, or submission requirements; what files must be run; what final files must be submitted; any grading criteria, restrictions, or important details that could cause marks to be lost. After fully understanding the assignment, create a clear internal plan and then complete the entire assignment according to those instructions. Important requirements: 1. Follow the PDF instructions exactly. Do not invent requirements or skip anything. 2. Use the provided files as the main source of truth. 3. Complete every required question, task, calculation, code section, analysis, and output. 4. If coding is required, inspect the existing project/files before making changes. 5. Do not randomly rewrite or modify working code. Change only what is required by the assignment. 6. Make sure the code actually runs and produces the required results. 7. Validate the final results against the assignment instructions. 8. Check for missing sections, incorrect calculations, incomplete answers, formatting problems, or required evidence before finishing. 9. Keep explanations clear, simple, and appropriate for a student assignment. 10. Do not leave placeholders such as “TODO,” “add screenshot,” “complete later,” or unfinished sections unless the assignment requires information that genuinely cannot be obtained from the provided materials. 11. Do not stop after partially completing the work. Finish the assignment in one complete pass as far as the provided materials allow. Before giving me the final answer, perform a final checklist against every instruction in the PDF and make sure nothing has been missed. At the end, give me a clear summary with: A. What you completed. B. How to run it. C. What results I should expect. D. What to submit. E. Final verification.

Output of the AI tool

- Question 1: the full architecture, assumptions, equations, operations and uncertainty text, and the two diagrams (source: `Question1/make_diagram.py`, `report/q1_text.py`).
- Question 2: all code (`Question2/mm.py, checks.py, flow_fig1.py, train.py, analysis.py, select_d.py, run_all.sh`), the training runs, all tables and figures, the answers Q2.1–Q2.17, including the two predictions written before the experiments (Q2.5, Q2.11), which the tool wrote and committed to the repository before running the experiments they refer to.
- The PDF builder (`report/build_report.py`) that takes every number from the JSON files in `Question2/results/`.

Edits made

- Edits by the AI tool during the session: fixed the edge count (6, not 7) in the Q1 diagram and text after noticing the mismatch; no results were edited by hand.
- **Edits by the student:** (to be filled in by the student after reviewing the report — list any changes made to text, code or numbers).